

Designing and Documenting Models: The ODD Protocol

EES 4760/5760

Agent-Based and Individual-Based Computational Modeling

J. Magnolia Gilligan

Class #4: Tuesday, September 8, 2026



Getting Started:

- Purpose of Today's Class:
 - Learn a structure for designing and documenting a model
- Getting started for today:
 - Download and save the Butterfly model from https://ees4760.jmgilligan.org/models/class_04/butterfly_odd.nlogox
 - Or go to the "Downloads" page at <https://ees4760.jmgilligan.org> and click on item 4: "Butterfly Model ODD"
 - Open NetLogo and load "butterfly_odd.nlogox"

Designing and Documenting Models

Designing and Documenting Models

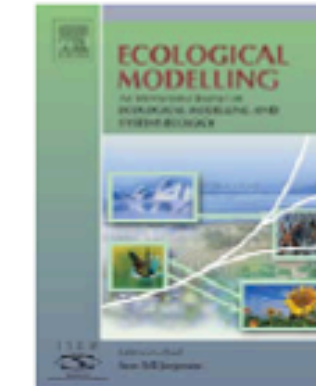
ECOLOGICAL MODELLING 198 (2006) 115–126



available at www.sciencedirect.com



journal homepage: www.elsevier.com/locate/ecolmodel



A standard protocol for describing individual-based and agent-based models

Volker Grimm^{a,*}, Uta Berger^b, Finn Bastiansen^a, Sigrunn Eliassen^c, Vincent Ginot^d, Jarl Giske^c, John Goss-Custard^e, Tamara Grand^f, Simone K. Heinz^c, Geir Huse^g, Andreas Huth^a, Jane U. Jepsen^a, Christian Jørgensen^c, Wolf M. Mooij^h, Birgit Müller^a, Guy Pe'erⁱ, Cyril Piou^b, Steven F. Railsback^j, Andrew M. Robbins^k, Martha M. Robbins^k, Eva Rossmanith^l, Nadja Rüger^a, Espen Strand^c, Sami Souissi^m, Richard A. Stillman^e, Rune Vabø^g, Ute Visser^a, Donald L. DeAngelisⁿ

^a UFZ Umweltforschungszentrum Leipzig-Halle GmbH, Department Ökologische Systemanalyse, Permoserstr. 15, 04318 Leipzig, Germany

^b Zentrum für Marine Tropenökologie, Fahrenheitstr. 6, 28359 Bremen, Germany

^c University of Bergen, Department of Biology, P.O. Box 7800, N-5020 Bergen, Norway

^d INRA, Unité de Biométrie, Domaine St.-Paul, 84 814 Avignon Cedex 9, France

Design

- Don't start writing code until you know what you're trying to do.
- Big picture
 - What is the **purpose** of your model?
 - What **things** does your model use?
 - How do those things **behave**?
- Design concepts
 - How do you represent the **things** in your model?
 - How do you implement their **behavior**?
 - How will your **things** and **behaviors** realize your **purpose**?
 - What **data** will you collect from your model?
 - How will you **use that data** to achieve your **purpose**?

Overview, Design Concepts, and Details

Elements of the ODD protocol	
Overview	1. Purpose and patterns
	2. Entities, state variables, and scales
	3. Process overview and scheduling
Design concepts	4. Design concepts • Basic principles • Emergence • Adaptation • Objectives • Learning • Prediction • Sensing • Interaction • Stochasticity • Collectives • Observation
Details	5. Initialization
	6. Input data
	7. Submodels

General



Detailed

ODD in perspective:

- Write overview and major parts of design concepts *first*
- As you write the model code, revisit and revise design concepts.
- Much of the details will emerge in the course of programming.
- When you are finished, write a complete ODD. This will be the major documentation for your model.

ODD Outline

1. Purpose & Patterns

Questions:

- What is the **purpose** of the model?
 - What are you trying to do?
 - ***Suggestion:*** What will the *key figure* showing your results look like?
- **Patterns:** What aspects of the world are we modeling?
 - How will you judge whether the model is realistic enough for the purpose?
 - What can you neglect?

2. Entities, State Variables, Scale

- What kinds of **entities** are in the model?
Agents, collectives, spatial units, global environment, ...
 - Turtles, breeds, patches, links, etc.
- What attributes (state-variables) characterize the entities?
Age, sex, wealth, mood, opinion, soil type, land costs, rainfall, market price, ...
 - `turtles-own`, `patches-own`, etc.
 - You only need state-variables if they vary
 - values are different for different entities, change over time, etc.
 - Use the fewest attributes you need
- What are the temporal and spatial resolutions and extents of the model?
 - **Resolution:** how detailed?
 - **Extents:** how big?
 - “Model settings” button in NetLogo

3. Process Overview and Scheduling

- How do states change?
- What entities do what, and in what order?
 - Schedule:
 1. Which entities take actions?
 2. What actions do they take?
 - Details of actions often go in **submodels**
 - e.g., `to-go` used submodels `to-eat` & `to-move` in our sugarscape model
 3. In what order do they take them?
 4. Design Concepts

4. Design Concepts

There are **11 design concepts** (see Table 3.1).

Textbook has one chapter for each.

Outline of Design Concepts

1. **Basic Principles:** Basis of model in general concepts and theories

2. **Emergence:** What emerges as the model runs?
(phenomena not imposed or directly programmed)

3. **Adaptation** How do agents respond to changes in their environment?
What decisions do they make, and how do they decide?
Do they seek objectives directly (*deliberately*) or indirectly (*mimic natural behavior*)?

Note: *Adaptation* means responding to current conditions. It's different from *persistent change* (the way the word is used in *evolution*)

Seek shelter when it starts to rain.

4. **Objectives (Fitness):** Goals of agents? What determines survival?
Do objectives change as agent changes?

5. **Learning:** How do individuals change behavior as they gain experience?

Note: This is where *persistent change* happens. It's often confused with *adaptation*

After getting rained on several times, start carrying an umbrella when it's cloudy.

Outline of Design Concepts (cont.)

6. **Prediction:** How do agents predict consequences of their decisions?
(learning, memory, environmental cues, programmed assumptions)
7. **Sensing:** What do agents know or perceive when making decisions?
(Is sensing process itself explicitly modeled, or do they *just know*?)
8. **Interaction:** What forms of interaction among agents are there?
9. **Stochasticity:** Is there randomness in model? *Randomness must be justified!*
10. **Collectives:** Grouping of individuals (herds, families, teams, social networks, ...)
11. **Observation:** How do scientists collect data from the model for analysis?

Details

5. Initialization

- What is the initial state of the model world?
 - Time $t = 0$ of a simulation run
- In detail:
 - How many entities, of what type, are there initially?
 - What are the exact values of their state variables?
(Or how were they set at random?)
 - Is initialization always the same,
or does it vary from one simulation run to the next?
 - Are initial values chosen arbitrarily, or based on data?
 - References to those data should be provided.

6. Input data

- Does the model use input from external sources to represent processes that change over time?
 - data files, other models, human interaction
- If so, what data?
 - Where did they come from?
 - Provide references, citations.

7. Submodels

If the **process scheduling** step contains a list of processes or actions, explain, in detail what **submodels** represent those processes or actions.

- What are the model parameters?
- How were the submodels designed or chosen?
- How were they tested?

Example: Virtual Corridors for Conservation Management

Example: Virtual Corridors for Conservation Management

Research Notes

Virtual Corridors for Conservation Management

GUY PE'ER,^{*†‡} DAVID SALTZ,[†] AND KARIN FRANK^{*}

^{*}Department of Ecological Modelling, UFZ-Centre for Environmental Research Leipzig-Halle, Permoserstrasse 15, 04318 Leipzig, Germany

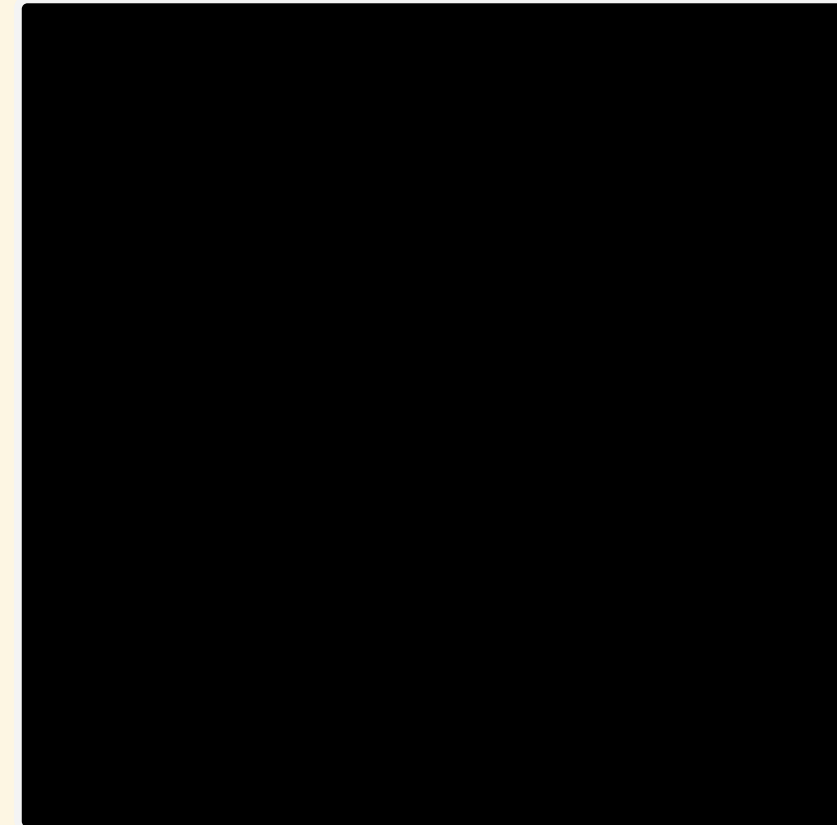
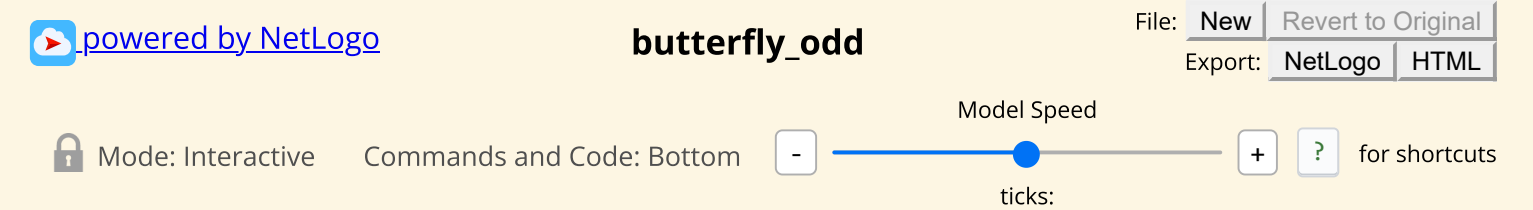
[†]Mitrani Department for Desert Ecology, Ben-Gurion University of the Negev, Sde Boker Campus, 84990 Sde Boker, Israel

Abstract: *Corridors are usually perceived as clearly visible, linear landscape elements embedded in a hostile environment that connect two or more larger blocks of habitat. Animal response to certain aspects of landscape heterogeneity, however, can channel their movements into specific routes that may appear similar to their surroundings. These routes can be described as "virtual corridors" (VCs). Here we contribute to the foundation of the concept of VCs and highlight their implications for conservation management. We used an individual-based model to analyze the formation of VCs in the case of hilltopping in butterflies—where males and virgin females ascend to hilltops and mate. We simulated butterfly movements in two different topographically heterogeneous landscapes. We analyzed the movement patterns with respect to one parameter, the intensity of response to topography. Virtual corridor structure depended on the behavioral parameter, landscape, and location of the source patch. Within a realistic range of the behavioral parameter and in a realistic landscape, VC structures may be complex and require individual-based models for their elucidation.*

Key Words: habitat gradients, hilltopping, individual-based model, landscape heterogeneity, landscape management, nonrandom dispersal, topography

Butterfly Model in NetLogo

- Open NetLogo and load “butterfly_odd.nlogo”
- Code section is blank, but ODD is filled in on “Info” tab.
 - As you read Chapter 4, for Thursday, the book will guide you to fill in the code based on ODD, as you read.
 - You don’t need to turn this in, but you will use it as the basis for homework due Sept. 16.
- Click on “Edit” (pencil icon) to see what Info tab looks like when you edit it.
 - For details on editing “Info” tab, open [NetLogo User Guide](#) from the NetLogo Help menu and go to “Info Tab Guide” in the “Reference Section”



Command Center	▼
NetLogo Code	▼
Model Info	▼
Agent Inspection	▼

Purpose

- Ecologists observe that as butterflies move uphill, they concentrate into narrow and well-defined *virtual corridors* rather than following any old path to the top of the hill.
- Explore the concept of *virtual corridors*:
Can concentrations of migrating animals emerge spontaneously from movement behavior and topography, instead of being a special habitat?
- Specifically, How does the concentration of hill-topping butterflies emerge from:
 - The way butterflies move uphill
 - The topography of the landscape

Entities, State Variables, and Scales

- **Landscape:**
 - Square grid cells (patches) with one *state variable*: **elevation**.
- **Butterflies:**
 - Have one *state variable*: **location**
(discrete: which patch they're in)

Entities, State Variables, and Scales

- **Spatial Scale:**

- 150×150 cells (patches)
- Two modes:
 - Generic mode: No specific size for a patch
 - Mapping mode: Each patch corresponds to 25×25 meters in the real landscape

- **Time Scale:**

- Simulations last 1000 ticks
- Tick length is unspecified (time for a butterfly to move one patch).

Process Overview and Scheduling

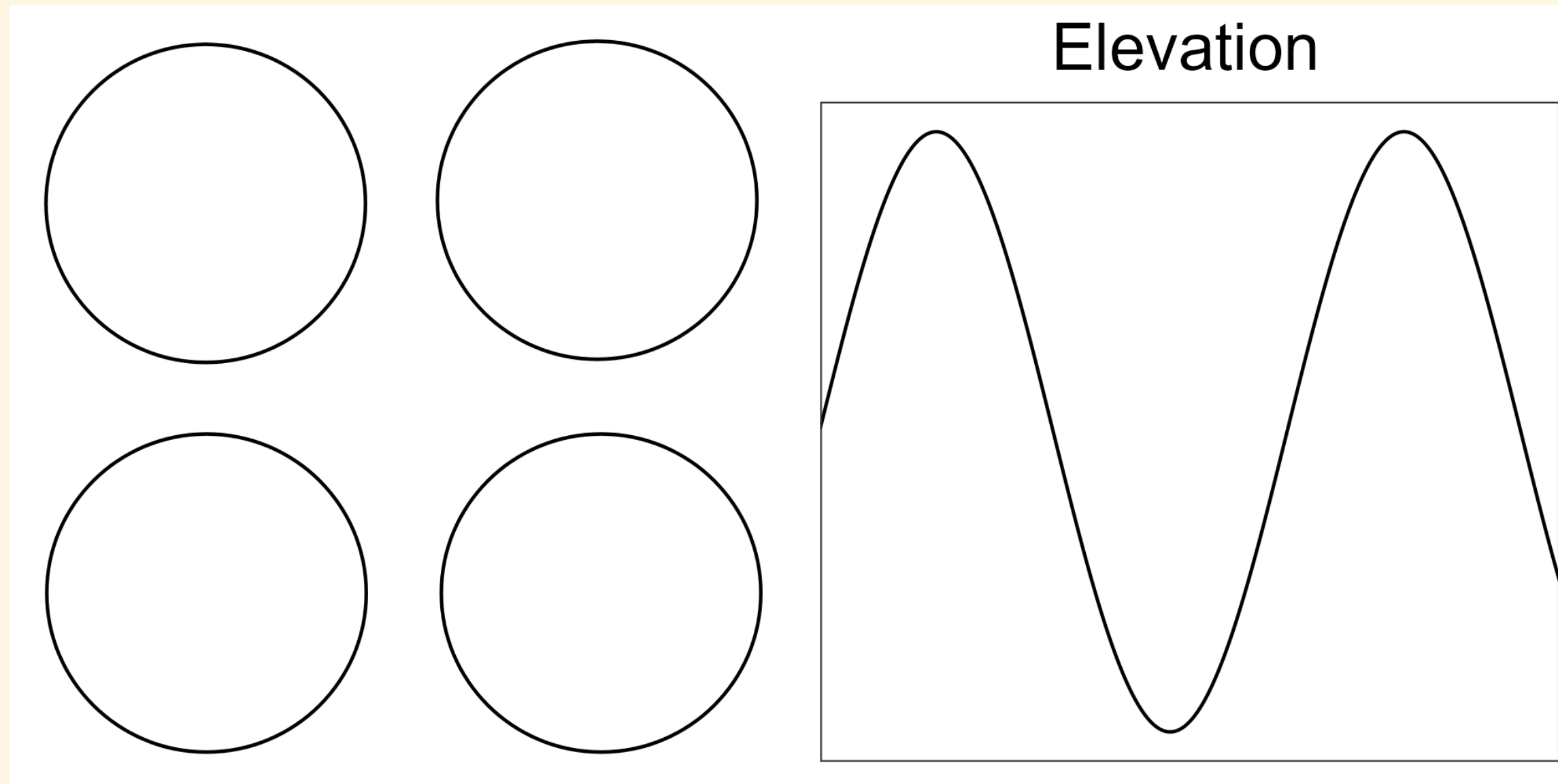
- **Only one process:** butterfly movement
 - On each tick, each butterfly moves once
 - The order in which butterflies move is not important because they don't interact

Design concepts (important ones)

- **Basic Principles:** The concept of *virtual corridors*
- **Emergence:** results (concentration of butterflies in corridors)
emerge from movement rule and topography
- **Sensing:** Butterflies can sense elevation in current and 8 surrounding patches
- **Interaction:** None
- **Stochasticity:** Used to represent reasons why butterflies do not move straight uphill
- **Observation:** We need a way to measure of butterfly concentration

Initialization

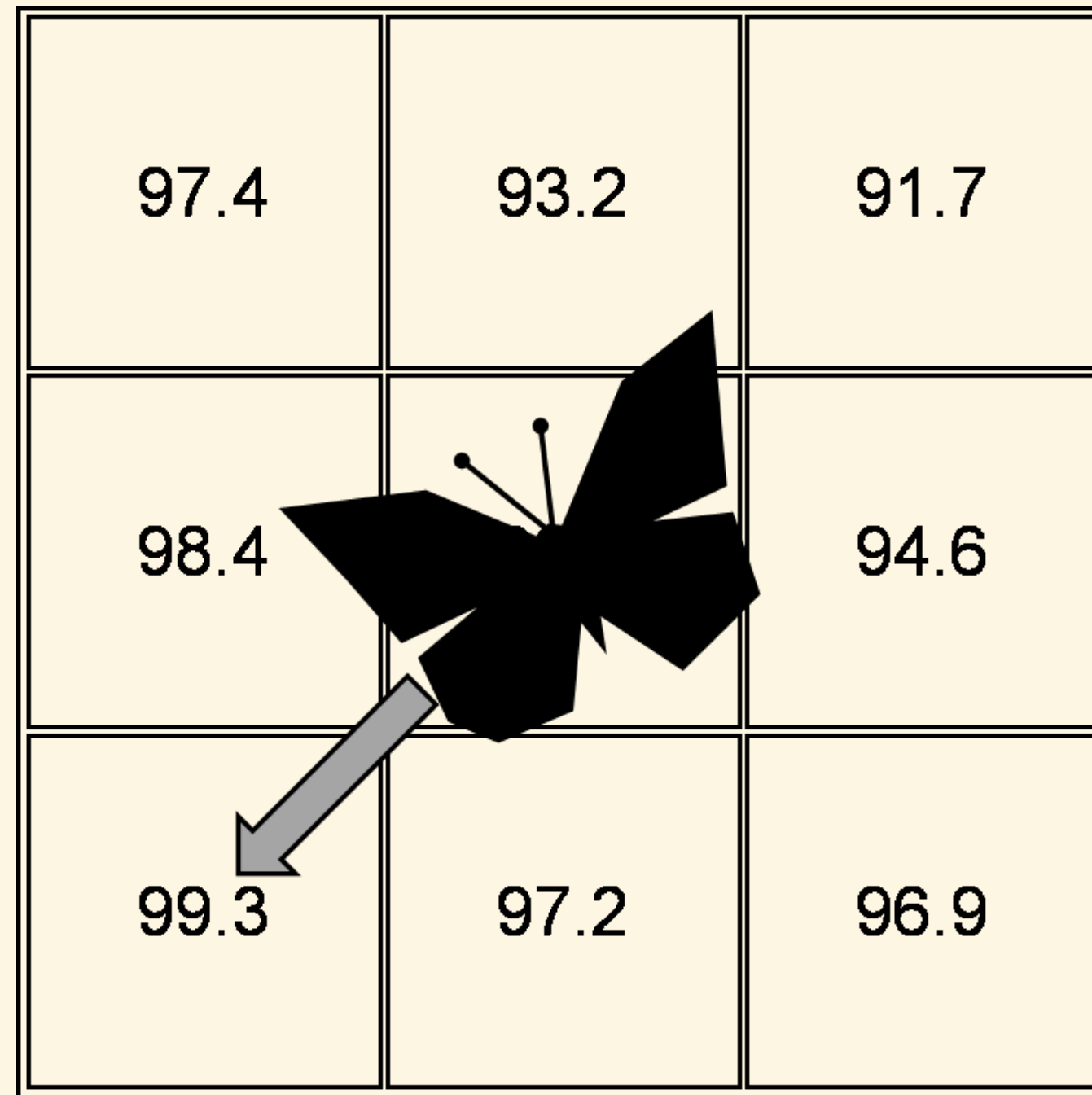
- **Landscape:** cell elevations set to a flat landscape with four hills



- **Butterflies:** 500 are created and placed in one patch

Submodel: Butterfly movement

- Global parameter q is probability that butterfly moves straight uphill, vs. moving to random neighbor cell.



Next Steps

Next Steps

For next week, you will follow the book and write code to implement the butterfly model.

Catching up

Catching up from Thursday

- Download the example model: Go to Downloads page, “Sugarscape model from class #3”, and
 - download “Netlogo model from class”,
 - or download directly from https://ees4760.jmgilligan.org/models/class_03/class_03_example-b.nlogox

Working with agentsets (*with*)

- *turtles* is an agentset of all turtles.
- *with* is one of many primitives that create a subset of an agentset:
`ask turtles with [color = blue] [move]`
- Similar keywords for subsetting:
 - *with-min*, *with-max* (one or more turtles)
 - *n-of*, *max-n-of*, *min-n-of* (exactly *n* turtles)
 - *one-of*, *max-one-of*, *min-one-of* (exactly one turtle)
- Use the dictionary to look up correct syntax.
- Click “setup” and then try this:

```
ask patches [ set pcolor black ]  
ask patches with-max [sugar] [ set pcolor cyan ]  
ask max-n-of 5 patches [sugar] [ set pcolor pink ]
```

Movement in our Model

```
to move
  if sugar <= 0 or any? other turtles-here
  [
    move-to max-one-of neighbors [ sugar ]
  ]
end
```

If there isn't any sugar, or if there's another turtle on the patch, it moves to the neighboring patch with the most sugar.

- We run `move` in the context of a turtle:
 - For each neighbor of the patch the turtle is on, look up `sugar` in the context of that neighbor
 - The brackets `[ ... ]` signal the change of context.
 - Find the neighbors with the maximum `sugar`, and pick one of them at random
 - In the context of the original turtle, move to that neighboring patch

Working with stochasticity (randomness)

- Uniform distribution of random integers less than `q`

```
random q
```

- `random 5` would return 0, 1, 2, 3, or 4

- Uniform distribution of floating-point numbers less than `x`

```
random-float q
```

- `random-float 5` would return a number between 0 and 4.9999.... The number could be very close to 5.

- Select one patch at random and turn it green

```
ask one-of patches [set pcolor green]
```

- Select one agent at random and turn it right 5 degrees:

```
ask one-of turtles [right 5]
```

Random Numbers

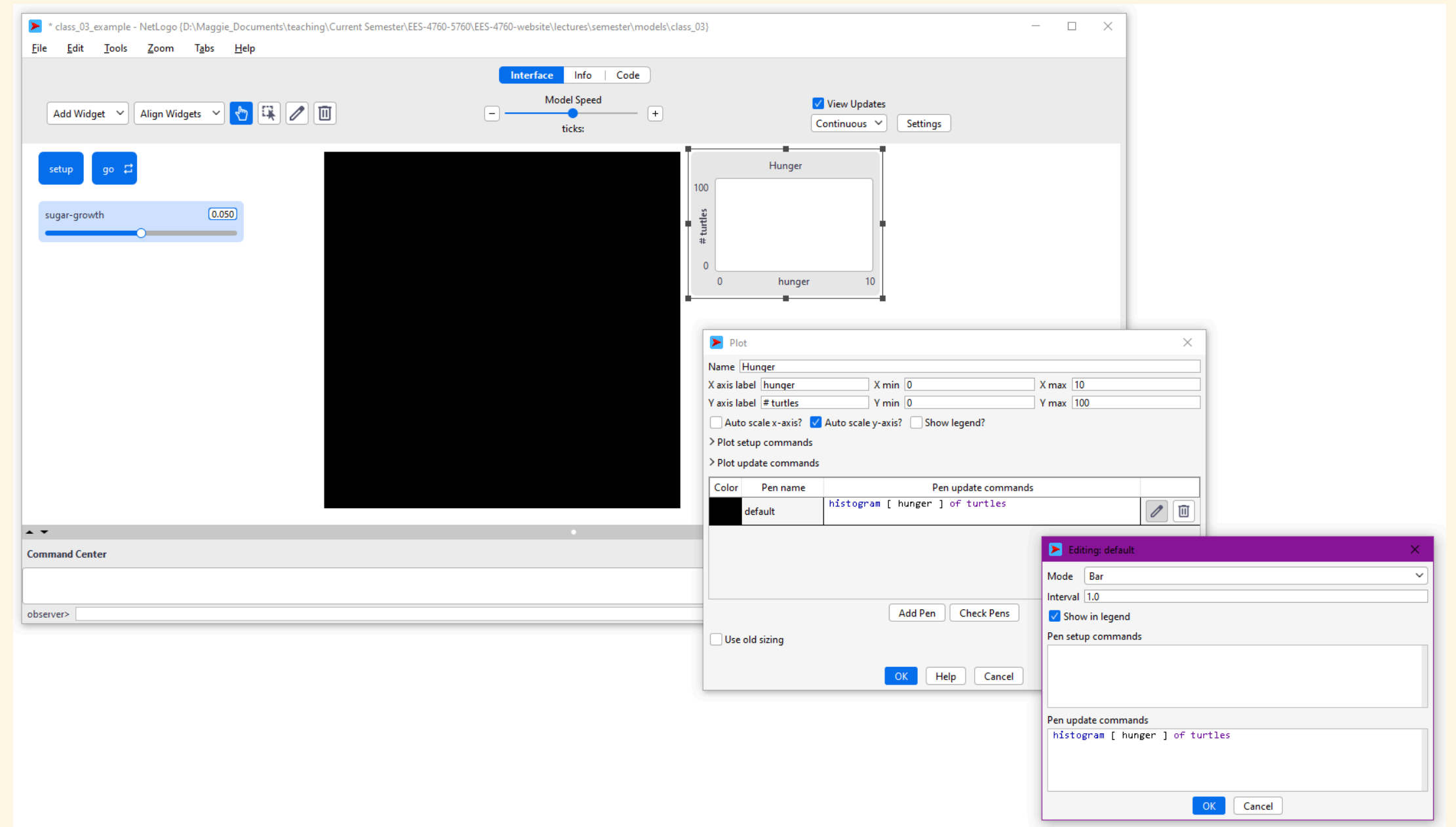
- Important: remember that
 - `random q` returns an integer n where $0 \leq n < q$
 - `random-float q` returns a floating-point number x where $0 \leq x < q$
 - If $q = 0.4$,
 - What does `random q` return?
 - What does `random-float q` return?
 - Should we worry that `random-float q` can't return q ?

```
let q 0.4
let x (list)
repeat 10000 [ set x lput (random-float q) x ]
show max x
show (q - max x) / q
```

Monitoring a Model

On the interface tab:

- Right click and add a Plot
 - Name the plot “Hunger”
 - Set X max to 10 and Y max to 100
 - Type “Hunger” for “X axis label” and “# Turtles” for “Y axis label”
 - Click on the pencil icon next to “default” pen
 - Choose “Bar” for “Mode”
 - In “Pen update commands” type `histogram [hunger] of turtles`
 - Press “OK”



Play with the model

- Do interesting things happen for different values of **sugar-growth**?
- It might be fun to comment out the line in **to go** that stops the model after 2000 ticks

```
; if ticks > 2000 [ stop ]
```

The image shows the NetLogo interface for a model named 'class_03_example-c'. At the top, there's a 'powered by NetLogo' logo and a 'File' menu with 'New' and 'Revert to Original' options. Below the menu, there's a 'Mode: Interactive' indicator and a 'Commands and Code: Bottom' label. A 'Model Speed' slider is set to 1, with a 'ticks:' label below it. On the left, there are 'setup' and 'go' buttons, and a 'sugar-growth' slider set to 0.05. The main area is a large black rectangle representing the model. On the right, there's a 'Hunger' plot showing the number of turtles (y-axis, 0 to 100) versus hunger (x-axis, 0 to 10). At the bottom, there's a 'Command Center' dropdown menu with options: 'Command Center', 'NetLogo Code', 'Model Info', and 'Agent Inspection'.

Command Center
NetLogo Code
Model Info
Agent Inspection

More sophisticated sugarscape model

- Download the “fancy sugarscape” model from the Class #4 downloads page https://ees4760.jmgilligan.org/models/class_03/fancy-sugarscape.nlogox
 - You will also need to download the data file with a map of the sugar distribution: https://ees4760.jmgilligan.org/models/class_03/sugar-map.txt
 - This is similar to the research model used by Axtell and Epstein to study theoretical economics.

More about Agent-Based Models

Agent-based models

- Agents/Individuals are discrete, unique, and autonomous entities.
 - Discrete entities: Important at low densities
 - Unique: Individuals, even of same age and species, can be **different**
 - Individuals have a **life history**
- Interactions among individuals are usually **local**, not global
- Individuals make decisions, which can be **adaptive**
- Ecology or society **emerges** from individual behavior (bottom-up)

Why agent-based models?

1. Individuals/agents are unique and different from one another
2. Individuals/agents interact locally
3. Individuals/agents show adaptive behavior

Why agent-based models?

Use ABM if *one or more* of the following are **essential** to your research question:

1. Individual variability
2. Local interactions
3. Adaptive behavior

- ABMs that include all three elements can be called **full-fledged**.
- Most ABMs focus on only one or two elements.

Why **not** agent-based models?

- Too complex
- Too data hungry.
- Too many parameters unknown.
- Too much uncertainty in model structure.
- Hard to test.
- Require too much person and computer power.

When ABMs are too hard, use aggregated modeling techniques:

- Microeconomics looks at aggregate supply and demand;
 - does not model individual consumers and producers.
- Biology can use population dynamics without looking at individuals
- Chemists model chemical reactions with rate constants,
 - not individual atoms and molecules.